

✅ RAG Learning Roadmap (0 to 100)

📖 Stage 1: Fundamentals (0–20%)

◆ 1. What is RAG?

- RAG = Retrieval + Generation
- Combines search-based methods (retrieval) with language generation models (like BART, T5)

◆ 2. Why RAG?

- Boosts factual accuracy
- Allows use of external knowledge bases
- Efficient for dynamic or domain-specific info

◆ 3. Use Cases

- Customer support bots
 - Document Q&A systems
 - Chatbots with custom knowledge
-

🧠 Stage 2: Components of RAG (20–40%)

◆ 1. Retriever

- Pulls relevant documents for a given query
- Common retrievers: BM25, DPR, FAISS, Elasticsearch

◆ 2. Generator

- Generates a final response using query + retrieved docs
- Models: BART, T5, GPT-3/4, etc.

◆ 3. Workflow

User Query → Retriever → Retrieved Documents → Generator → Final Answer

🔧 Stage 3: Tooling & Libraries (40–60%)

◆ Hugging Face Transformers

- Implements RAG via facebook/rag-token-base and others
- Uses RagRetriever, RagTokenForGeneration

◆ LangChain

- Framework for chaining LLMs + tools
- RAG = retriever + LLM chain

◆ Haystack

- End-to-end pipelines for search + generation
 - Easy integration with document stores, retrievers, generators
-

📁 Stage 4: Hands-On Projects (60–80%)

◆ Project 1: Basic RAG with Hugging Face

- Use RagRetriever + RagTokenForGeneration to build Q&A

◆ Project 2: RAG Chatbot with LangChain

- Use FAISS + OpenAI + your custom docs

◆ Project 3: Build a Q&A System with Haystack

- Upload PDF → Index → Ask Questions → Get Answers
-

🚀 Stage 5: Advanced RAG (80–100%)

◆ Customize RAG:

- Use your own documents (PDF, CSV, Web pages)
- Vector embeddings (OpenAI, Hugging Face, etc.)

◆ Optimization:

- Chunking strategies (how to break down documents)
- Re-ranking results
- Cost-effective deployment (use local models)

◆ Deployment:

- Use Streamlit/FastAPI for frontend
 - Host on cloud (e.g., Hugging Face Spaces, Render, etc.)
-

📁 Resources & Docs

- Hugging Face RAG Docs
- LangChain Docs
- Haystack Docs

Building a Simple RAG System using Hugging Face (with dummy data)

🔑 Step-by-Step: Basic RAG with Hugging Face

We'll create a **question-answering system** using the **RAG model (facebook/rag-token-base)**.

What You'll Learn:

1. Load the RAG model
 2. Use it to **retrieve documents**
 3. Generate an answer based on those documents
-

Step 1: Install Required Libraries

```
pip install transformers faiss-cpu torch
```

faiss-cpu is used for document similarity search.

Step 2: Code to Load Model & Ask a Question

```
from transformers import RagTokenizer, RagRetriever, RagTokenForGeneration
```

```
# Load tokenizer
```

```
tokenizer = RagTokenizer.from_pretrained("facebook/rag-token-base")
```

```
# Load retriever (using dummy index for now)
```

```
retriever = RagRetriever.from_pretrained(  
    "facebook/rag-token-base",  
    index_name="exact", # for real use-case, use "custom" or "legacy"  
    use_dummy_dataset=True # just to make it run without setting up real data  
)
```

```
# Load model
```

```
model = RagTokenForGeneration.from_pretrained("facebook/rag-token-base", retriever=retriever)
```

```
# Input query
```

```
input_text = "Who was the first person on the moon?"
```

```
# Tokenize input
```

```
inputs = tokenizer(input_text, return_tensors="pt")
```

```
# Generate answer
```

```
generated = model.generate(**inputs)
```

Decode output

```
answer = tokenizer.batch_decode(generated, skip_special_tokens=True)[0]  
print("Answer:", answer)
```

Output Example:

Answer: Neil Armstrong was the first person on the moon.

Summary of What You Did:

- Loaded a pretrained **RAG model**
 - Asked a question
 - It **retrieved** knowledge (from dummy data here)
 - It **generated** a human-like answer
-

Would you like to:

1. Try using **your own custom documents** (PDF, text)?
2. Use **LangChain** for a more modular RAG system?
3. Get an **assignment/project** to practice this?

Let's move to the **next practical milestone**:

Step 4B: Using Custom Documents in RAG with LangChain

Now you'll learn how to:

1. Load your **own documents** (PDFs, .txt, etc.)
 2. Convert them to **embeddings**
 3. Use them with a **retriever**
 4. Ask questions using a **RAG pipeline**
-

Prerequisites

Install LangChain + required libraries:

```
pip install langchain openai faiss-cpu tiktoken
```

You'll need an **OpenAI API key** (or I can show how to use a local model if needed)

Step 1: Create a Text File

Let's create a file my_knowledge.txt with this sample content:

Albert Einstein was a theoretical physicist who developed the theory of relativity.

He won the Nobel Prize in Physics in 1921 for his explanation of the photoelectric effect.

💡 Step 2: LangChain Code to Load and Ask Questions

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# 1. Load document
loader = TextLoader("my_knowledge.txt")
docs = loader.load()

# 2. Split text into chunks
splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
split_docs = splitter.split_documents(docs)

# 3. Convert to vector store
embeddings = OpenAIEmbeddings() # You'll need OPENAI_API_KEY env var
db = FAISS.from_documents(split_docs, embeddings)

# 4. Create retriever and QA chain
retriever = db.as_retriever()
qa_chain = RetrievalQA.from_chain_type(llm=OpenAI(), retriever=retriever)

# 5. Ask a question
query = "Who won the Nobel Prize in 1921?"
result = qa_chain.run(query)

print("Answer:", result)
```

✅ What You Just Learned:

- How to turn your **own data** into a **retrieval-ready knowledge base**

- How to **query that data** using RAG with LangChain
-

Progress Update: 90% Done!

Next up (for final 10%):

- Deploy this as a web app (e.g., Streamlit chatbot)
- Use local models instead of OpenAI (for full control)
- Optimize chunking, indexing, and embedding costs

You're at **90%**, so now we'll cover the **final step**:

Step 5: Deploy Your RAG App as a Chatbot with Streamlit

This will turn your LangChain + custom data pipeline into a **real-time chatbot** on your screen.

What You'll Build:

- Upload your own file
 - Ask questions from it
 - Get answers using RAG
 - All through a **simple web interface**
-

Prerequisites

Install Streamlit and LangChain:

```
pip install streamlit langchain openai faiss-cpu
```

Step-by-Step Streamlit RAG App

rag_app.py

```
import streamlit as st
```

```
from langchain.document_loaders import TextLoader
```

```
from langchain.text_splitter import CharacterTextSplitter
```

```
from langchain.vectorstores import FAISS
```

```
from langchain.embeddings import OpenAIEmbeddings
```

```
from langchain.chains import RetrievalQA
```

```
from langchain.llms import OpenAI
```

```
import os
```

```
st.title(" RAG Chatbot - Ask Your Files!")
```

```
uploaded_file = st.file_uploader("Upload a .txt file", type="txt")
```

```
if uploaded_file:
```

```
    with open("temp.txt", "wb") as f:
```

```
        f.write(uploaded_file.getbuffer())
```

```
    # Load + Split
```

```
    loader = TextLoader("temp.txt")
```

```
    docs = loader.load()
```

```
    splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
```

```
    split_docs = splitter.split_documents(docs)
```

```
    # Embeddings + Vector Store
```

```
    embeddings = OpenAIEmbeddings()
```

```
    db = FAISS.from_documents(split_docs, embeddings)
```

```
    retriever = db.as_retriever()
```

```
    # RAG Chain
```

```
    qa_chain = RetrievalQA.from_chain_type(llm=OpenAI(), retriever=retriever)
```

```
    # Ask Question
```

```
    question = st.text_input("Ask a question about the uploaded file:")
```

```
    if question:
```

```
        response = qa_chain.run(question)
```

```
        st.write("🤖 Answer:", response)
```



Run It:

```
streamlit run rag_app.py
```



Important:

You need to export your OpenAI key before running:

```
export OPENAI_API_KEY=your-key-here # for Mac/Linux
```

```
# or
```

set OPENAI_API_KEY=your-key-here # for Windows CMD

You Did It!

 You now have a working RAG chatbot:

- Upload any .txt file
 - Ask custom questions
 - Get answers in real time
-

You've Reached: 100% Mastery of RAG!

Want to go **beyond 100%**?

-  Use **PDFs or CSVs** instead of .txt
-  Switch to **local models** to reduce OpenAI cost
-  Deploy on Hugging Face Spaces or Render

Let me know and I'll help you build the advanced version 